Optimisation Methods for Variational Inference: Mathematical Solutions and Derivations

David Ewing (82171165)
ENEL445 – Applied Engineering Optimisation
University of Canterbury

March 17, 2026

Abstract

This document presents the complete mathematical solution for applying gradient-based optimisation methods to Variational Inference (VI) in Bayesian linear regression. We derive the Evidence Lower Bound (ELBO), present the baseline Coordinate Ascent VI (CAVI) solution, and develop gradient and Hessian expressions for advanced optimisation methods including gradient ascent, Newton's method, and BFGS. The formulation demonstrates how VI can serve as a realistic optimisation testbed for comparing methods from ENEL445.

Contents

1 Problem Formulation

1.1 Bayesian Linear Regression Model

Consider the standard linear regression model:

$$y_i = \boldsymbol{x}_i^T \boldsymbol{\beta} + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \tau_e^{-1}), \quad i = 1, \ldots, n \tag{1}$$

where:

- $\boldsymbol{y} \in \mathbb{R}^n$ is the response vector
- $\boldsymbol{X} \in \mathbb{R}^{n \times p}$ is the design matrix with rows $\boldsymbol{x}_i^T$
- $\boldsymbol{\beta} \in \mathbb{R}^p$ are regression coefficients
- $\tau_e > 0$ is the precision (inverse variance) of residuals

In vector form:

$$\boldsymbol{y} \sim \mathcal{N}(\boldsymbol{X}\boldsymbol{\beta}, \tau_e^{-1} \boldsymbol{I}_n) \tag{2}$$

1.2 Prior Distributions

We use conjugate Normal-Gamma priors:

$$p(\boldsymbol{\beta}) = \mathcal{N}(\boldsymbol{\beta} \mid \boldsymbol{0}, \lambda_\beta^{-1} \boldsymbol{I}_p) \tag{3}$$

$$p(\tau_e) = \text{Gamma}(\tau_e \mid \alpha_e, \gamma_e) \tag{4}$$

For flat (improper) priors, we set $\lambda_\beta \to 0$, $\alpha_e = \gamma_e = 0$.

1.3 Posterior Distribution

The joint posterior distribution is:

$$p(\boldsymbol{\beta}, \tau_e \mid \boldsymbol{y}) = \frac{p(\boldsymbol{y} \mid \boldsymbol{\beta}, \tau_e) p(\boldsymbol{\beta}) p(\tau_e)}{p(\boldsymbol{y})} \tag{5}$$

The normalizing constant $p(\boldsymbol{y}) = \int \int p(\boldsymbol{y}, \boldsymbol{\beta}, \tau_e)\, d\boldsymbol{\beta}\, d\tau_e$ is analytically tractable for this conjugate model, but we treat it as intractable to demonstrate VI methodology.

2 Variational Inference Framework

2.1 Mean-Field Approximation

Variational Inference approximates the posterior with a simpler distribution $q(\boldsymbol{\beta}, \tau_e)$ from a tractable family. We use the mean-field factorization:

$$q(\boldsymbol{\beta}, \tau_e) = q(\boldsymbol{\beta})\, q(\tau_e) \tag{6}$$

with Gaussian and Gamma factors:

$$q(\boldsymbol{\beta}) = \mathcal{N}(\boldsymbol{\beta} \mid \boldsymbol{\mu}_\beta, \boldsymbol{\Sigma}_\beta) \tag{7}$$

$$q(\tau_e) = \text{Gamma}(\tau_e \mid a_e, b_e) \tag{8}$$

2.2 Variational Parameters

The design variables (in ENEL445 terminology) are the variational parameters:

$$\boldsymbol{\phi} = (\boldsymbol{\mu}_\beta, \boldsymbol{\Sigma}_\beta, a_e, b_e) \tag{9}$$

Dimension count for p predictors:

- $\boldsymbol{\mu}_\beta$: p parameters
- $\boldsymbol{\Sigma}_\beta$: $p(p+1)/2$ parameters (symmetric matrix)
- a_e, b_e: 2 parameters
- **Total**: $d = p + \frac{p(p+1)}{2} + 2 = \frac{p(p+3)}{2} + 2$

For $p = 3$: $d = 11$ dimensions.

2.3 Evidence Lower Bound (ELBO)

The optimisation objective is the Evidence Lower Bound:

$$\text{ELBO}(\boldsymbol{\phi}) = \mathbb{E}_q[\log p(\boldsymbol{y}, \boldsymbol{\beta}, \tau_e)] - \mathbb{E}_q[\log q(\boldsymbol{\beta}, \tau_e)] \tag{10}$$

The ELBO is a lower bound on the log marginal likelihood:

$$\log p(\boldsymbol{y}) = \text{ELBO}(\boldsymbol{\phi}) + \text{KL}(q \| p) \geq \text{ELBO}(\boldsymbol{\phi}) \tag{11}$$

since $\text{KL}(q\|p) \geq 0$. Maximizing the ELBO minimizes the reverse KL divergence $\text{KL}(q\|p)$.

3 ELBO Derivation

3.1 Joint Log-Likelihood Expansion

The expected joint log-likelihood decomposes as:

$$\mathbb{E}_q[\log p(\boldsymbol{y}, \boldsymbol{\beta}, \tau_e)] = \mathbb{E}_q[\log p(\boldsymbol{y} \mid \boldsymbol{\beta}, \tau_e)] + \mathbb{E}_q[\log p(\boldsymbol{\beta})] + \mathbb{E}_q[\log p(\tau_e)] \tag{12}$$

3.2 Likelihood Term

The log-likelihood is:

$$\log p(\boldsymbol{y} \mid \boldsymbol{\beta}, \tau_e) = -\frac{n}{2}\log(2\pi) + \frac{n}{2}\log \tau_e - \frac{\tau_e}{2}\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|^2 \tag{13}$$

Taking expectations under q:

$$\begin{aligned}\mathbb{E}_q[\log p(\boldsymbol{y} \mid \boldsymbol{\beta}, \tau_e)] = &-\frac{n}{2}\log(2\pi) + \frac{n}{2}\mathbb{E}_q[\log \tau_e] \\ &- \frac{1}{2}\mathbb{E}_q[\tau_e]\mathbb{E}_q[\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|^2]\end{aligned} \tag{14}$$

Using the mean-field factorization:

$$\begin{aligned}\mathbb{E}_q[\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|^2] &= \mathbb{E}_q[(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta})^T(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta})] \\ &= \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu}_\beta\|^2 + \text{tr}(\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\Sigma}_\beta)\end{aligned} \tag{15}$$

For Gamma(a_e, b_e):

$$\mathbb{E}_q[\tau_e] = \frac{a_e}{b_e} \tag{16}$$

$$\mathbb{E}_q[\log \tau_e] = \psi(a_e) - \log b_e \tag{17}$$

where $\psi(\cdot)$ is the digamma function.
Therefore:

$$\begin{aligned}\mathbb{E}_q[\log p(\boldsymbol{y} \mid \boldsymbol{\beta}, \tau_e)] = &-\frac{n}{2}\log(2\pi) + \frac{n}{2}(\psi(a_e) - \log b_e) \\ &- \frac{a_e}{2b_e}\left[\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu}_\beta\|^2 + \operatorname{tr}(\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\Sigma}_\beta)\right]\end{aligned} \tag{18}$$

3.3 Prior Terms

For the beta prior (assuming flat prior $\lambda_\beta = 0$):

$$\mathbb{E}_q[\log p(\boldsymbol{\beta})] = \text{constant} \tag{19}$$

For the Gamma prior:

$$\begin{aligned}\mathbb{E}_q[\log p(\tau_e)] &= (\alpha_e - 1)\mathbb{E}_q[\log \tau_e] - \gamma_e \mathbb{E}_q[\tau_e] + \alpha_e \log \gamma_e - \log \Gamma(\alpha_e) \\ &= (\alpha_e - 1)(\psi(a_e) - \log b_e) - \gamma_e \frac{a_e}{b_e} + C\end{aligned} \tag{20}$$

3.4 Entropy Terms

The entropy of the variational distribution:

$$H[q] = -\mathbb{E}_q[\log q(\boldsymbol{\beta}, \tau_e)] = H[q(\boldsymbol{\beta})] + H[q(\tau_e)] \tag{21}$$

For $q(\boldsymbol{\beta}) = \mathcal{N}(\boldsymbol{\mu}_\beta, \boldsymbol{\Sigma}_\beta)$:

$$H[q(\boldsymbol{\beta})] = \frac{p}{2}(1 + \log 2\pi) + \frac{1}{2}\log|\boldsymbol{\Sigma}_\beta| \tag{22}$$

For $q(\tau_e) = \text{Gamma}(a_e, b_e)$:

$$H[q(\tau_e)] = a_e - \log b_e + \log \Gamma(a_e) + (1 - a_e)\psi(a_e) \tag{23}$$

3.5 Complete ELBO Expression

Combining all terms:

$$\begin{aligned}\text{ELBO}(\boldsymbol{\phi}) = &-\frac{n}{2}\log(2\pi) + \frac{n}{2}(\psi(a_e) - \log b_e) \\ &- \frac{a_e}{2b_e}\left[\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu}_\beta\|^2 + \operatorname{tr}(\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\Sigma}_\beta)\right] \\ &+ (\alpha_e - 1)(\psi(a_e) - \log b_e) - \gamma_e \frac{a_e}{b_e} \\ &+ \frac{p}{2}(1 + \log 2\pi) + \frac{1}{2}\log|\boldsymbol{\Sigma}_\beta| \\ &+ a_e - \log b_e + \log \Gamma(a_e) + (1 - a_e)\psi(a_e) + C\end{aligned} \tag{24}$$

4 Coordinate Ascent Variational Inference (CAVI)

4.1 Algorithm Overview

CAVI updates each variational parameter (or block) while holding others fixed, cycling until convergence. This is analogous to block coordinate descent/ascent in ENEL445 Chapter 4.

Algorithm 1 Coordinate Ascent VI

1: Initialise $\boldsymbol{\mu}_\beta^{(0)}, \boldsymbol{\Sigma}_\beta^{(0)}, a_e^{(0)}, b_e^{(0)}$
2: **for** $t = 0, 1, 2, \ldots$ until convergence **do**
3: Update $q(\boldsymbol{\beta})$ given current $q(\tau_e)$
4: Update $q(\tau_e)$ given current $q(\boldsymbol{\beta})$
5: Check convergence: $\max_i |\phi_i^{(t+1)} - \phi_i^{(t)}|/|\phi_i^{(t+1)}| < \epsilon$
6: **end for**

4.2 Update for $q(\boldsymbol{\beta})$

Taking functional derivative of ELBO with respect to $q(\boldsymbol{\beta})$ and setting to zero:

$$\log q^*(\boldsymbol{\beta}) = \mathbb{E}_{q(\tau_e)}[\log p(\boldsymbol{y}, \boldsymbol{\beta}, \tau_e)] + \text{const} \tag{25}$$

This yields a Gaussian distribution:

$$q^*(\boldsymbol{\beta}) = \mathcal{N}(\boldsymbol{\beta} \mid \boldsymbol{\mu}_\beta, \boldsymbol{\Sigma}_\beta) \tag{26}$$

$$\boldsymbol{\Sigma}_\beta = \left(\lambda_\beta \boldsymbol{I} + \mathbb{E}_q[\tau_e] \boldsymbol{X}^T \boldsymbol{X}\right)^{-1} = \left(\frac{a_e}{b_e} \boldsymbol{X}^T \boldsymbol{X}\right)^{-1} \tag{27}$$

$$\boldsymbol{\mu}_\beta = \mathbb{E}_q[\tau_e] \boldsymbol{\Sigma}_\beta \boldsymbol{X}^T \boldsymbol{y} = \frac{a_e}{b_e} \boldsymbol{\Sigma}_\beta \boldsymbol{X}^T \boldsymbol{y} \tag{28}$$

For flat prior ($\lambda_\beta = 0$).

4.3 Update for $q(\tau_e)$

Similarly, taking functional derivative with respect to $q(\tau_e)$:

$$\log q^*(\tau_e) = \mathbb{E}_{q(\boldsymbol{\beta})}[\log p(\boldsymbol{y}, \boldsymbol{\beta}, \tau_e)] + \text{const} \tag{29}$$

This yields a Gamma distribution:

$$q^*(\tau_e) = \text{Gamma}(\tau_e \mid a_e, b_e) \tag{30}$$

$$a_e = \alpha_e + \frac{n}{2} \tag{31}$$

$$b_e = \gamma_e + \frac{1}{2}\left[\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu}_\beta\|^2 + \text{tr}(\boldsymbol{X}^T \boldsymbol{X} \boldsymbol{\Sigma}_\beta)\right] \tag{32}$$

4.4 CAVI Properties

- **Monotonic convergence**: ELBO increases at each iteration
- **Linear convergence rate**: Error decreases geometrically
- **Computational cost**: $O(np^2)$ per iteration (dominated by $(\boldsymbol{X}^T \boldsymbol{X})^{-1}$)
- **Limitation**: Does not exploit joint parameter updates; slow for coupled parameters

5 Gradient-Based Optimisation

5.1 Gradient Ascent with Line Search

Instead of coordinate updates, compute the full gradient $\nabla_{\boldsymbol{\phi}}\text{ELBO}(\boldsymbol{\phi})$ and update jointly:

$$\boldsymbol{\phi}^{(t+1)} = \boldsymbol{\phi}^{(t)} + \alpha^{(t)}\nabla_{\boldsymbol{\phi}}\text{ELBO}(\boldsymbol{\phi}^{(t)}) \tag{33}$$

where $\alpha^{(t)}$ is determined by backtracking line search satisfying Armijo condition:

$$\text{ELBO}(\boldsymbol{\phi}^{(t)} + \alpha\boldsymbol{d}^{(t)}) \geq \text{ELBO}(\boldsymbol{\phi}^{(t)}) + c_1\alpha\nabla\text{ELBO}(\boldsymbol{\phi}^{(t)})^T\boldsymbol{d}^{(t)} \tag{34}$$

with $\boldsymbol{d}^{(t)} = \nabla\text{ELBO}(\boldsymbol{\phi}^{(t)})$ and $c_1 = 10^{-4}$.

5.2 Gradient Derivations

We parameterize $\boldsymbol{\Sigma}_\beta$ via Cholesky decomposition $\boldsymbol{\Sigma}_\beta = \boldsymbol{L}\boldsymbol{L}^T$ to maintain positive definiteness, or work directly with precision $\boldsymbol{\Lambda}_\beta = \boldsymbol{\Sigma}_\beta^{-1}$.

5.2.1 Gradient w.r.t. $\boldsymbol{\mu}_\beta$

From equation (24), the terms depending on $\boldsymbol{\mu}_\beta$ are:

$$\text{ELBO} \supset -\frac{a_e}{2b_e}\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu}_\beta\|^2 \tag{35}$$

Taking the gradient:

$$\begin{aligned}\nabla_{\boldsymbol{\mu}_\beta}\text{ELBO} &= -\frac{a_e}{2b_e}\nabla_{\boldsymbol{\mu}_\beta}[(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu}_\beta)^T(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu}_\beta)] \\ &= -\frac{a_e}{2b_e}\cdot 2\boldsymbol{X}^T(\boldsymbol{X}\boldsymbol{\mu}_\beta - \boldsymbol{y}) \\ &= \frac{a_e}{b_e}\boldsymbol{X}^T(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu}_\beta)\end{aligned} \tag{36}$$

This is $\mathbb{E}_q[\tau_e]$ times the negative gradient of least squares objective.

5.2.2 Gradient w.r.t. $\boldsymbol{\Sigma}_\beta$

Terms depending on $\boldsymbol{\Sigma}_\beta$:

$$\text{ELBO} \supset -\frac{a_e}{2b_e}\text{tr}(\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\Sigma}_\beta) + \frac{1}{2}\log|\boldsymbol{\Sigma}_\beta| \tag{37}$$

Using matrix calculus identities:

$$\nabla_{\boldsymbol{\Sigma}_\beta}\text{tr}(\boldsymbol{A}\boldsymbol{\Sigma}_\beta) = \boldsymbol{A}^T \tag{38}$$

$$\nabla_{\boldsymbol{\Sigma}_\beta}\log|\boldsymbol{\Sigma}_\beta| = \boldsymbol{\Sigma}_\beta^{-1} \tag{39}$$

Therefore:

$$\begin{aligned}\nabla_{\boldsymbol{\Sigma}_\beta}\text{ELBO} &= -\frac{a_e}{2b_e}\boldsymbol{X}^T\boldsymbol{X} + \frac{1}{2}\boldsymbol{\Sigma}_\beta^{-1} \\ &= \frac{1}{2}\left(\boldsymbol{\Sigma}_\beta^{-1} - \frac{a_e}{b_e}\boldsymbol{X}^T\boldsymbol{X}\right)\end{aligned} \tag{40}$$

Note: Since $\boldsymbol{\Sigma}_\beta$ is symmetric, we account for this in implementation.

5.2.3 Gradient w.r.t. a_e

Terms depending on a_e:

$$\begin{aligned}\text{ELBO} \supset & \frac{n}{2}\psi(a_e) - \frac{a_e}{2b_e}\left[\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu}_\beta\|^2 + \text{tr}(\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\Sigma}_\beta)\right] \\ & + (\alpha_e - 1)\psi(a_e) - \gamma_e\frac{a_e}{b_e} + a_e + (1 - a_e)\psi(a_e) + \log\Gamma(a_e) \end{aligned} \tag{41}$$

Simplifying (using $\frac{d}{da}\log\Gamma(a) = \psi(a)$):

$$\begin{aligned}\nabla_{a_e}\text{ELBO} &= \frac{n}{2}\psi'(a_e) - \frac{1}{2b_e}\left[\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu}_\beta\|^2 + \text{tr}(\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\Sigma}_\beta)\right] \\ &\quad + (\alpha_e - 1)\psi'(a_e) - \frac{\gamma_e}{b_e} + 1 - \psi(a_e) + (1 - a_e)\psi'(a_e) + \psi(a_e) \\ &= \left(\frac{n}{2} + \alpha_e - a_e\right)\psi'(a_e) - \frac{1}{2b_e}\left[\text{SSR} + \text{tr}(\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\Sigma}_\beta)\right] - \frac{\gamma_e}{b_e} + 1 \end{aligned} \tag{42}$$

where $\text{SSR} = \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu}_\beta\|^2$ and $\psi'(\cdot)$ is the trigamma function.

5.2.4 Gradient w.r.t. b_e

Terms depending on b_e:

$$\begin{aligned}\text{ELBO} \supset & -\frac{n}{2}\log b_e + \frac{a_e}{2b_e}\left[\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu}_\beta\|^2 + \text{tr}(\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\Sigma}_\beta)\right] \\ & - (\alpha_e - 1)\log b_e + \gamma_e\frac{a_e}{b_e} - \log b_e \end{aligned} \tag{43}$$

Taking derivative:

$$\begin{aligned}\nabla_{b_e}\text{ELBO} &= -\frac{n}{2b_e} - \frac{a_e}{2b_e^2}\left[\text{SSR} + \text{tr}(\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\Sigma}_\beta)\right] - \frac{\alpha_e - 1}{b_e} - \frac{\gamma_e a_e}{b_e^2} - \frac{1}{b_e} \\ &= -\frac{1}{b_e}\left(\frac{n}{2} + \alpha_e\right) - \frac{a_e}{b_e^2}\left(\frac{\text{SSR} + \text{tr}(\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\Sigma}_\beta)}{2} + \gamma_e\right) \end{aligned} \tag{44}$$

5.3 Gradient Ascent Properties

- **Convergence rate**: Linear (geometric) for strongly convex objectives
- **Computational cost**: $O(np^2)$ per gradient evaluation + line search overhead
- **Advantage**: Exploits full gradient, can handle parameter coupling
- **Disadvantage**: May be slower than CAVI for conjugate models where updates are analytic

6 Newton's Method

6.1 Newton Update

Newton's method uses second-order information:

$$\boldsymbol{\phi}^{(t+1)} = \boldsymbol{\phi}^{(t)} - [\boldsymbol{H}(\boldsymbol{\phi}^{(t)})]^{-1}\nabla\text{ELBO}(\boldsymbol{\phi}^{(t)}) \tag{45}$$

where $\boldsymbol{H} = \nabla^2\text{ELBO}$ is the Hessian matrix. For maximization, we use $-\boldsymbol{H}^{-1}$ to ascend.

6.2 Hessian Derivations

The full Hessian is a $(d \times d)$ matrix with blocks:

$$\boldsymbol{H} = \begin{bmatrix} \nabla^2_{\boldsymbol{\mu}_\beta,\boldsymbol{\mu}_\beta} & \nabla^2_{\boldsymbol{\mu}_\beta,\boldsymbol{\Sigma}_\beta} & \nabla^2_{\boldsymbol{\mu}_\beta,a_e} & \nabla^2_{\boldsymbol{\mu}_\beta,b_e} \\ \nabla^2_{\boldsymbol{\Sigma}_\beta,\boldsymbol{\mu}_\beta} & \nabla^2_{\boldsymbol{\Sigma}_\beta,\boldsymbol{\Sigma}_\beta} & \nabla^2_{\boldsymbol{\Sigma}_\beta,a_e} & \nabla^2_{\boldsymbol{\Sigma}_\beta,b_e} \\ \nabla^2_{a_e,\boldsymbol{\mu}_\beta} & \nabla^2_{a_e,\boldsymbol{\Sigma}_\beta} & \nabla^2_{a_e,a_e} & \nabla^2_{a_e,b_e} \\ \nabla^2_{b_e,\boldsymbol{\mu}_\beta} & \nabla^2_{b_e,\boldsymbol{\Sigma}_\beta} & \nabla^2_{b_e,a_e} & \nabla^2_{b_e,b_e} \end{bmatrix} \tag{46}$$

6.2.1 Block: $\nabla^2_{\boldsymbol{\mu}_\beta,\boldsymbol{\mu}_\beta}$

From equation (36):

$$\begin{aligned} \nabla^2_{\boldsymbol{\mu}_\beta,\boldsymbol{\mu}_\beta}\text{ELBO} &= \nabla_{\boldsymbol{\mu}_\beta}\left[\frac{a_e}{b_e}\boldsymbol{X}^T(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu}_\beta)\right] \\ &= -\frac{a_e}{b_e}\boldsymbol{X}^T\boldsymbol{X} \end{aligned} \tag{47}$$

This is negative definite (since $\boldsymbol{X}^T\boldsymbol{X} \succ 0$ for full rank $\boldsymbol{X}$), indicating concavity in $\boldsymbol{\mu}_\beta$.

6.2.2 Block: $\nabla^2_{\boldsymbol{\Sigma}_\beta,\boldsymbol{\Sigma}_\beta}$

From equation (40), using vectorization:

$$\nabla^2_{\boldsymbol{\Sigma}_\beta,\boldsymbol{\Sigma}_\beta}\text{ELBO} = -\frac{1}{2}(\boldsymbol{\Sigma}_\beta^{-1} \otimes \boldsymbol{\Sigma}_\beta^{-1}) \tag{48}$$

where $\otimes$ denotes Kronecker product. This is negative definite on the symmetric matrix subspace.

6.2.3 Mixed Blocks

Cross-derivatives are generally non-zero, coupling the parameters:

$$\nabla^2_{\boldsymbol{\mu}_\beta,b_e}\text{ELBO} = -\frac{a_e}{b_e^2}\boldsymbol{X}^T(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu}_\beta) \tag{49}$$

$$\nabla^2_{\boldsymbol{\mu}_\beta,a_e}\text{ELBO} = \frac{1}{b_e}\boldsymbol{X}^T(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\mu}_\beta) \tag{50}$$

$$\nabla^2_{a_e,a_e}\text{ELBO} = \left(\frac{n}{2} + \alpha_e - a_e\right)\psi''(a_e) \tag{51}$$

$$\nabla^2_{b_e,b_e}\text{ELBO} = \frac{1}{b_e^2}\left(\frac{n}{2} + \alpha_e\right) + \frac{2a_e}{b_e^3}\left(\frac{\text{SSR} + \text{tr}(\boldsymbol{X}^T\boldsymbol{X}\boldsymbol{\Sigma}_\beta)}{2} + \gamma_e\right) \tag{52}$$

where $\psi''(\cdot)$ is the second derivative of digamma (tetragamma).

6.3 Newton's Method Properties

- **Convergence rate**: Quadratic near optimum ($\|\boldsymbol{\phi}^{(t+1)} - \boldsymbol{\phi}^*\| = O(\|\boldsymbol{\phi}^{(t)} - \boldsymbol{\phi}^*\|^2)$)
- **Computational cost**: $O(d^3)$ for Hessian inversion per iteration
- **Challenge**: Hessian may not be positive definite away from optimum
- **Safeguard**: Add regularization $\boldsymbol{H} + \lambda\boldsymbol{I}$ or use trust region

For $p = 3$, $d = 11$, Hessian inversion costs $\approx 1,300$ flops – manageable.

7 BFGS Quasi-Newton Method

7.1 BFGS Update

BFGS approximates the (inverse) Hessian using gradient differences:

$$\boldsymbol{B}_{t+1} = \boldsymbol{B}_t + \frac{\boldsymbol{y}_t \boldsymbol{y}_t^T}{\boldsymbol{y}_t^T \boldsymbol{s}_t} - \frac{\boldsymbol{B}_t \boldsymbol{s}_t \boldsymbol{s}_t^T \boldsymbol{B}_t}{\boldsymbol{s}_t^T \boldsymbol{B}_t \boldsymbol{s}_t} \tag{53}$$

where:

$$\boldsymbol{s}_t = \boldsymbol{\phi}^{(t+1)} - \boldsymbol{\phi}^{(t)} \quad \text{(step)} \tag{54}$$

$$\boldsymbol{y}_t = \nabla \text{ELBO}(\boldsymbol{\phi}^{(t+1)}) - \nabla \text{ELBO}(\boldsymbol{\phi}^{(t)}) \quad \text{(gradient change)} \tag{55}$$

and $\boldsymbol{B}_t \approx -\boldsymbol{H}^{-1}$ (inverse Hessian for maximization).

7.2 BFGS Search Direction

The search direction is:

$$\boldsymbol{d}^{(t)} = \boldsymbol{B}_t \nabla \text{ELBO}(\boldsymbol{\phi}^{(t)}) \tag{56}$$

Step size α_t is determined by line search (Wolfe conditions).

7.3 BFGS Properties

- **Convergence rate**: Superlinear (between linear and quadratic)
- **Computational cost**: $O(d^2)$ per iteration (vs $O(d^3)$ for Newton)
- **Advantage**: No Hessian computation, maintains positive definiteness automatically
- **Limitation**: Requires good line search, memory for storing $\boldsymbol{B}_t$ or history (L-BFGS variant)

BFGS is often the best general-purpose optimiser for smooth unconstrained problems.

8 Constrained Optimisation via Penalty Methods

8.1 Constraint Formulation

To address variance collapse, impose minimum variance constraints:

$$\text{maximize} \quad \text{ELBO}(\boldsymbol{\phi}) \tag{57}$$

$$\text{subject to} \quad \sigma^2_{\beta_j} \geq \sigma^2_{\min}, \quad j = 1, \ldots, p \tag{58}$$

$$\tau_e^{-1} \geq \sigma^2_{e,\min} \tag{59}$$

where $\sigma^2_{\beta_j} = [\boldsymbol{\Sigma}_\beta]_{jj}$ are diagonal elements.

8.2 Quadratic Penalty Method

Convert to unconstrained problem:

$$\text{maximize} \quad \text{ELBO}(\boldsymbol{\phi}) - \rho \sum_{j=1}^{p} \max(0, \sigma^2_{\min} - \sigma^2_{\beta_j})^2 \tag{60}$$

Solve sequence with increasing $\rho_k \to \infty$:

Algorithm 2 Penalty Method for Constrained VI

Initialise $\boldsymbol{\phi}^{(0)}$, $\rho_0 > 0$, $\tau > 1$
for $k = 0, 1, 2, \ldots$ **do**
    Solve unconstrained: $\boldsymbol{\phi}^{(k+1)} = \arg\max_{\boldsymbol{\phi}} [\text{ELBO}(\boldsymbol{\phi}) - \rho_k P(\boldsymbol{\phi})]$
    Update penalty: $\rho_{k+1} = \tau \rho_k$
    Check feasibility and convergence
end for

8.3 Penalty Method Properties

- **Advantage**: Converts constrained to unconstrained problem
- **Disadvantage**: Ill-conditioning as $\rho \to \infty$ (condition number $\kappa \sim O(\rho)$)
- **Alternative**: Augmented Lagrangian (better conditioning)

9 Computational Complexity Summary

Method	Per Iteration	Convergence Rate	Hessian?
CAVI	$O(np^2)$	Linear	No
Gradient Ascent	$O(np^2)$+ line search	Linear	No
Newton	$O(np^2 + d^3)$	Quadratic	Yes
BFGS	$O(np^2 + d^2)$	Superlinear	Approx
Penalty	(Inner loop cost) $\times K$	Linear per subproblem	Depends

Table 1: Computational complexity comparison for VI optimisation methods. n = sample size, p = predictors, $d = p(p+3)/2 + 2$ = total variational parameters, K = penalty iterations.

10 Summary and Recommendations

10.1 Key Results

1. **ELBO derivation**: Complete analytical expression for Bayesian linear regression VI
2. **CAVI solution**: Closed-form coordinate ascent updates (equations 27–32)
3. **Gradient expressions**: Full gradient for all variational parameters (equations 36–44)
4. **Hessian blocks**: Second derivatives for Newton's method (equation 47 and subsequent)
5. **BFGS formulation**: Quasi-Newton updates without explicit Hessian
6. **Constrained VI**: Penalty method formulation for minimum variance constraints

10.2 Expected Performance

- **CAVI**: Baseline – stable, monotonic, but may be slow for coupled parameters
- **Gradient Ascent**: Better than CAVI if parameter coupling strong; requires good line search

- **Newton**: Fastest convergence near optimum but expensive per iteration and may need regularization
- **BFGS**: Best overall balance – superlinear convergence without Hessian computation
- **Penalty**: Addresses under-dispersion but adds outer loop iterations

10.3 Implementation Strategy

1. Implement and validate CAVI against analytical posteriors
2. Add gradient computation with numerical checks ($\|\nabla f - \nabla f_{\text{numerical}}\| < 10^{-6}$)
3. Implement gradient ascent with backtracking line search
4. Add full Hessian computation and Newton's method with regularization
5. Implement BFGS using scipy.optimize.minimize or custom code
6. Compare all methods on test problems varying (n, p)
7. Extend to penalty method if time permits

10.4 Validation Checks

- **ELBO monotonicity**: Should increase each iteration (for CAVI, may not hold for Newton without line search)
- **Gradient accuracy**: Numerical gradient check with finite differences
- **Hessian accuracy**: Test on analytical cases, check symmetry
- **Convergence to analytical posterior**: For conjugate model with flat priors
- **Posterior calibration**: Coverage probabilities for credible intervals

11 Conclusion

This document provides the complete mathematical solution for applying ENEL445 optimisation methods to Variational Inference. The ELBO maximisation problem offers a realistic testbed with:

- Smooth, high-dimensional objective function
- Analytical gradient and Hessian computations
- Natural baseline (CAVI) for comparison
- Extension to constrained optimisation
- Validation against analytical posteriors

The derivations demonstrate that VI is fundamentally an optimisation problem suitable for systematic comparison of gradient-based methods, connecting Bayesian inference to engineering optimisation coursework.

A Special Functions

A.1 Digamma Function

$$\psi(x) = \frac{d}{dx}\log\Gamma(x) = \frac{\Gamma'(x)}{\Gamma(x)} \tag{61}$$

Series expansion for large x:

$$\psi(x) \approx \log x - \frac{1}{2x} - \frac{1}{12x^2} + O(x^{-4}) \tag{62}$$

A.2 Trigamma Function

$$\psi'(x) = \frac{d^2}{dx^2}\log\Gamma(x) \tag{63}$$

Series expansion:

$$\psi'(x) \approx \frac{1}{x} + \frac{1}{2x^2} + \frac{1}{6x^3} + O(x^{-4}) \tag{64}$$

B Matrix Calculus Identities

$$\nabla_{\boldsymbol{x}} \boldsymbol{x}^T \boldsymbol{A}\boldsymbol{x} = (\boldsymbol{A} + \boldsymbol{A}^T)\boldsymbol{x} \tag{65}$$

$$\nabla_{\boldsymbol{X}} \mathrm{tr}(\boldsymbol{A}\boldsymbol{X}) = \boldsymbol{A}^T \tag{66}$$

$$\nabla_{\boldsymbol{X}} \log|\boldsymbol{X}| = \boldsymbol{X}^{-T} = (\boldsymbol{X}^{-1})^T \tag{67}$$

$$\nabla_{\boldsymbol{X}} \mathrm{tr}(\boldsymbol{X}^{-1}\boldsymbol{A}) = -\boldsymbol{X}^{-T}\boldsymbol{A}\boldsymbol{X}^{-T} \tag{68}$$

For symmetric $\boldsymbol{X}$, account for off-diagonal elements appearing twice.

References

[1] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

[2] D. M. Blei, A. Kucukelbir, and J. D. McAuliffe, "Variational Inference: A Review for Statisticians," *Journal of the American Statistical Association*, vol. 112, no. 518, pp. 859–877, 2017.

[3] J. R. R. A. Martins and A. Ning, *Engineering Design Optimisation*, Cambridge University Press, 2021.

[4] J. Nocedal and S. J. Wright, *Numerical Optimisation*, 2nd ed., Springer, 2006.

[5] A. Gelman et al., *Bayesian Data Analysis*, 3rd ed., Chapman & Hall/CRC, 2013.